```
"use client";
import { useState, useEffect, useRef } from "react";
import Image from "next/image";
import { db } from "../lib/firebase";
import { ref, onValue, set, update, remove, get } from "firebase/database";

type PaketTuru = "trial" | "basic" | "pro";

interface PaketOzellikleri {
  maxMasa: number;
  maxGarson: number;
  qrkod: boolean;
  mutfakEkrani: boolean;
  sureSayaci: boolean;
  sesliUyari: boolean;
}

const PAKET_LIMITLERI: Record<PaketTuru, PaketOzellikleri> = {
  trial: {
    maxMasa: 5,
    maxGarson: 1,
    qrkod: false,
    mutfakEkrani: false,
    sureSayaci: false,
    sesliUyari: false,
  },
  basic: {
```

```
    maxMasa: 10,
    maxGarson: 2,
    qrkod: true,
    mutfakEkrani: false,
    sureSayaci: false,
    sesliUyari: true,
  },
  pro: {
    maxMasa: 50,
    maxGarson: 20,
    qrkod: true,
    mutfakEkrani: true,
    sureSayaci: true,
    sesliUyari: true,
  },
};

function SiparisSuresi({ eklenmeZamani }: { eklenmeZamani?: number }) {
  const [gecenSure, setGecenSure] = useState(0);

  useEffect(() => {
    if (!eklenmeZamani) return;
    const fark = Math.floor((Date.now() - eklenmeZamani) / 1000);
    setGecenSure(fark > 0 ? fark : 0);

    const sayac = setInterval(() => {
      const yeniFark = Math.floor((Date.now() - eklenmeZamani) / 1000);
```

```
    setGecenSure(yeniFark > 0 ? yeniFark : 0);
  }, 1000);

  return () => clearInterval(sayac);
 }, [eklenmeZamani]);

 const dakika = Math.floor(gecenSure / 60);
 const saniye = gecenSure % 60;
 const sureGectiMi = gecenSure > 600;

 return (
  <div
   className={`px-2 py-1 sm:px-3 sm:py-1.5 rounded-xl text-xs sm:text-sm font-mono font-extrabold flex items-center gap-1 shadow-md ${
     sureGectiMi
      ? "bg-red-500/35 text-red-300 border border-red-500 animate-pulse"
      : "bg-slate-800 text-slate-100 border border-slate-600"
   }`}
  >
   <span>
    {dakika}:{saniye < 10 ? `0${saniye}` : saniye}
   </span>
  </div>
 );
}

export default function Home() {
 const [isletmeKodu, setIsletmeKodu] = useState("");
```

```
const [isletmeSifre, setIsletmeSifre] = useState("");
const [aktifIsletme, setAktifIsletme] = useState<string | null>(null);
const [superAdminModu, setSuperAdminModu] = useState(false);
const [tumIsletmeler, setTumIsletmeler] = useState<any[]>([]);

const [superAdminModalAcik, setSuperAdminModalAcik] = useState(false);
const [superAdminSifreGiris, setSuperAdminSifreGiris] = useState("");

const [yeniIsletmeAdi, setYeniIsletmeAdi] = useState("");
const [yeniIsletmeSifre, setYeniIsletmeSifre] = useState("");

const [lisansKilitli, setLisansKilitli] = useState(false);
const [kalanGun, setKalanGun] = useState<number | null>(null);
const [aktifPaket, setAktifPaket] = useState<PaketTuru>("trial");
const [abonelikPanelAcik, setAbonelikPanelAcik] = useState(false);

const mevcutLimitler = PAKET_LIMITLERI[aktifPaket] || PAKET_LIMITLERI.trial;
const oncekiMasalarRef = useRef<any[]>([]);

const [kullaniciRolu, setKullaniciRolu] = useState<string | null>(null);
const [adminSifre, setAdminSifre] = useState("");
const [sifreModalAcik, setSifreModalAcik] = useState(false);
const [aktifSekme, setAktifSekme] = useState("salon");

const [masalar, setMasalar] = useState<any[]>([]);
const [menu, setMenu] = useState<any[]>([]);
const [stoklar, setStoklar] = useState<any[]>([]);
```

```
const [kategoriler] = useState([
  "Tümü",
  "Sıcak İçecekler",
  "Soğuk İçecekler",
  "Yiyecekler",
  "Tatlılar",
]);

const [kasa, setKasa] = useState({
  toplamCiro: 0,
  nakitToplam: 0,
  kartToplam: 0,
  kapatilanAdisyonSayisi: 0,
});

const [siparisler, setSiparisler] = useState<any[]>([]);
const [seciliQrMasa, setSeciliQrMasa] = useState<any>(null);
const [seciliMasa, setSeciliMasa] = useState<any>(null);
const [seciliKategori, setSeciliKategori] = useState("Tümü");
const [odemeYontemi, setOdemeYontemi] = useState("nakit");
const [masaNotu, setMasaNotu] = useState("");
const [iskontoOrani, setIskontoOrani] = useState(0);

const [yeniUrun, setYeniUrun] = useState({
  ad: "",
  fiyat: "",
  kategori: "Sıcak İçecekler",
```

```
});

const [yeniMasaAdi, setYeniMasaAdi] = useState("");
const [yeniStok, setYeniStok] = useState({
  ad: "",
  miktar: "",
  birim: "gr",
  kritikMiktar: "",
});

useEffect(() => {
  if (!superAdminModu) return;
  const isletmelerRef = ref(db, "isletmeler");
  const unsub = onValue(isletmelerRef, (snapshot) => {
    if (snapshot.exists()) {
      const data = snapshot.val();
      const liste = Object.keys(data).map((key) => ({
        kod: key,
        ...data[key],
      }));
      setTumIsletmeler(liste);
    } else {
      setTumIsletmeler([]);
    }
  });
  return () => unsub();
}, [superAdminModu]);
```

```
const urunSil = async (urunId: string) => {
  if (!aktifIsletme) return;
  if (confirm("Bu ürünü menüden silmek istediğinize emin misiniz?")) {
    const guncelMenu = menu.filter((u) => String(u.id) !== String(urunId));
    setMenu(guncelMenu);
    await set(ref(db, `isletmeler/${aktifIsletme}/menu`), guncelMenu);
  }
};

const calSiparisSesi = () => {
  if (!mevcutLimitler.sesliUyari) return;
  try {
    const AudioContext = window.AudioContext || (window as any).webkitAudioContext;
    if (!AudioContext) return;
    const ctx = new AudioContext();
    const osc1 = ctx.createOscillator();
    const gain1 = ctx.createGain();
    osc1.type = "sine";
    osc1.frequency.setValueAtTime(880, ctx.currentTime);
    gain1.gain.setValueAtTime(0.3, ctx.currentTime);
    gain1.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.4);
    osc1.connect(gain1);
    gain1.connect(ctx.destination);
    osc1.start();
    osc1.stop(ctx.currentTime + 0.4);
    setTimeout(() => {
```

```
      try {
        const osc2 = ctx.createOscillator();
        const gain2 = ctx.createGain();
        osc2.type = "sine";
        osc2.frequency.setValueAtTime(1320, ctx.currentTime);
        gain2.gain.setValueAtTime(0.3, ctx.currentTime);
        gain2.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.4);
        osc2.connect(gain2);
        gain2.connect(ctx.destination);
        osc2.start();
        osc2.stop(ctx.currentTime + 0.4);
      } catch (e) {}
    }, 200);
  } catch (err) {
    console.error("Ses çalınamadı:", err);
  }
};

useEffect(() => {
  if (!aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const lisansRef = ref(db, `${basePath}/lisans`);
  const masalarRef = ref(db, `${basePath}/masalar`);
  const kasaRef = ref(db, `${basePath}/kasa`);
  const menuRef = ref(db, `${basePath}/menu`);
  const stoklarRef = ref(db, `${basePath}/stoklar`);
  const siparislerRef = ref(db, `${basePath}/siparisler`);
```

```
const unsubLisans = onValue(lisansRef, (snapshot) => {
  if (snapshot.exists()) {
    const data = snapshot.val();
    const bugun = new Date();
    const bitis = new Date(data.bitisTarihi);
    const paketStr = String(data.aktifPaket || "").toLowerCase();
    if (paketStr.includes("pro")) setAktifPaket("pro");
    else if (paketStr.includes("basic")) setAktifPaket("basic");
    else setAktifPaket("trial");

    const farkZaman = bitis.getTime() - bugun.getTime();
    const farkGun = Math.ceil(farkZaman / (1000 * 3600 * 24));
    setKalanGun(farkGun);
    setLisansKilitli(bugun > bitis || data.durum === "pasif");
  }
});

const unsubMasalar = onValue(masalarRef, (snapshot) => {
  const data = snapshot.val();
  const maxMasaSayisi = mevcutLimitler?.maxMasa ?? 5;
  const hamVeri = snapshot.exists() && data ? data : {};

  const temizMasalarDizisi = Array.from({ length: maxMasaSayisi }, (_, index) => {
    const hedefId = String(index + 1);
    const kayitliMasa = hamVeri[hedefId] || Object.values(hamVeri).find(
      (m: any) => String(m?.id) === hedefId
```

```
);

if (kayitliMasa) {
  const adisyonDizi = Array.isArray((kayitliMasa as any).adisyon)
    ? (kayitliMasa as any).adisyon
    : [];
  const hesaplananTutar = adisyonDizi.reduce(
    (acc: number, item: any) =>
      acc + (Number(item.fiyat) || 0) * (Number(item.adet) || 0),
    0,
  );
  const netTutar =
    hesaplananTutar * (1 - (Number((kayitliMasa as any).iskonto) || 0) / 100);

  return {
    ...(kayitliMasa as object),
    id: hedefId,
    ad: (kayitliMasa as any).ad || `Masa ${hedefId}`,
    adisyon: adisyonDizi,
    tutar: netTutar,
    durum: adisyonDizi.length > 0 ? "dolu" : "bos",
  };
} else {
  return {
    id: hedefId,
    ad: `Masa ${hedefId}`,
    durum: "bos",
```

```
      tutar: 0,
      brutTutar: 0,
      iskonto: 0,
      not: "",
      adisyon: [],
      eklenmeZamani: null,
    };
  }
});

if (oncekiMasalarRef.current.length > 0) {
  let yeniSiparisVarMi = false;
  temizMasalarDizisi.forEach((yeniMasa: any) => {
    const eskiMasa = oncekiMasalarRef.current.find(
      (m: any) => String(m.id) === String(yeniMasa.id),
    );
    const yeniAdet = (yeniMasa.adisyon || []).reduce(
      (a: number, c: any) => a + (Number(c.adet) || 0),
      0,
    );
    const eskiAdet = (eskiMasa?.adisyon || []).reduce(
      (a: number, c: any) => a + (Number(c.adet) || 0),
      0,
    );
    if (yeniAdet > eskiAdet) yeniSiparisVarMi = true;
  });
```

```
    if (yeniSiparisVarMi && mevcutLimitler?.sesliUyari) {
      calSiparisSesi();
    }
  }

  oncekiMasalarRef.current = temizMasalarDizisi;
  setMasalar(temizMasalarDizisi);

  setSeciliMasa((prevSecili: any) => {
    if (!prevSecili) return null;
    const guncelMasa = temizMasalarDizisi.find(
      (m: any) => String(m.id) === String(prevSecili.id),
    );
    return guncelMasa || prevSecili;
  });
});

const unsubSiparisler = onValue(siparislerRef, (snapshot) => {
  const data = snapshot.val();
  if (data) {
    const yuklenenSiparisler = Object.keys(data)
      .map((key) => ({
        id: String(key),
        ...data[key],
      }))
      .filter((s: any) => s && s.durum === "bekliyor");
    setSiparisler(yuklenenSiparisler);
```

```
  } else {
    setSiparisler([]);
  }
});

const unsubKasa = onValue(kasaRef, (snapshot) => {
  const data = snapshot.val();
  if (data) setKasa(data);
});

const unsubMenu = onValue(menuRef, (snapshot) => {
  const data = snapshot.val();
  if (data)
    setMenu(Array.isArray(data) ? data.filter(Boolean) : Object.values(data));
});

const unsubStoklar = onValue(stoklarRef, (snapshot) => {
  const data = snapshot.val();
  if (data)
    setStoklar(Array.isArray(data) ? data.filter(Boolean) : Object.values(data));
});

return () => {
  unsubLisans();
  unsubMasalar();
  unsubSiparisler();
  unsubKasa();
```

```
    unsubMenu();
    unsubStoklar();
  };
}, [aktifIsletme, mevcutLimitler?.sesliUyari]);

useEffect(() => {
  if (seciliMasa) {
    setMasaNotu(seciliMasa.not || "");
    setIskontoOrani(seciliMasa.iskonto || 0);
  }
}, [seciliMasa?.id]);

const adminGirisYap = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!adminSifre) return;
  const systemRef = ref(db, "system/superAdminSifre");
  const systemSnapshot = await get(systemRef);
  const kayitliSuperSifre = systemSnapshot.exists() ? systemSnapshot.val() : "asdf";

  const isletmeRef = ref(db, `isletmeler/${aktifIsletme}/sifre`);
  const isletmeSnapshot = await get(isletmeRef);
  const kayitliIsletmeSifre = isletmeSnapshot.exists() ? isletmeSnapshot.val() : null;

  if (
    adminSifre === kayitliSuperSifre ||
    (kayitliIsletmeSifre && adminSifre === kayitliIsletmeSifre)
  ) {
```

```
    setKullaniciRolu("admin");
    setAktifSekme("salon");
    setSifreModalAcik(false);
    setAdminSifre("");
  } else {
    alert("Hatalı Yönetici Şifresi!");
    setAdminSifre("");
  }
};

const superAdminGirisKontrol = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!superAdminSifreGiris) return;
  const systemRef = ref(db, "system/superAdminSifre");
  const snapshot = await get(systemRef);
  const gercekSifre = snapshot.exists() ? snapshot.val() : "asdf";
  if (superAdminSifreGiris === gercekSifre) {
    setSuperAdminModu(true);
    setSuperAdminModalAcik(false);
    setSuperAdminSifreGiris("");
  } else {
    alert("Hatalı Süper Admin Şifresi!");
    setSuperAdminSifreGiris("");
  }
};

const isletmeGirisYap = (e: React.FormEvent) => {
```

```
  e.preventDefault();
  const kod = isletmeKodu.trim().toLowerCase().replace(/\s+/g, "-");
  if (!kod || !isletmeSifre) {
    alert("Lütfen işletme kodu ve şifresini giriniz!");
    return;
  }
  const isletmeRef = ref(db, `isletmeler/${kod}`);
  get(isletmeRef).then((snapshot) => {
    if (snapshot.exists()) {
      const data = snapshot.val();
      if (data.sifre === isletmeSifre) {
        setAktifIsletme(kod);
      } else {
        alert("Hatalı İşletme Şifresi!");
      }
    } else {
      alert("Bu kodla kayıtlı bir işletme bulunamadı! Süper Admin panelinden yeni işletme oluşturabilirsiniz.");
    }
  });
 };


 const yeniIsletmeOlustur = async (e: React.FormEvent) => {
  e.preventDefault();
  const kod = yeniIsletmeAdi.trim().toLowerCase().replace(/\s+/g, "-");
  if (!kod || !yeniIsletmeSifre) {
    alert("İşletme adı ve şifresi zorunludur!");
    return;
```

```
  }
  const defaultBitis = new Date();
  defaultBitis.setDate(defaultBitis.getDate() + 14);
  await set(ref(db, `isletmeler/${kod}`), {
    sifre: yeniIsletmeSifre,
    lisans: {
      aktifPaket: "Trial",
      bitisTarihi: defaultBitis.toISOString(),
      durum: "aktif",
    },
  });
  setYeniIsletmeAdi("");
  setYeniIsletmeSifre("");
  alert(`"${kod}" işletmesi başarıyla oluşturuldu!`);
};

const urunEkle = async (urun: any) => {
  if (!seciliMasa || seciliMasa.id === undefined || seciliMasa.id === null || !aktifIsletme) {
    alert("Lütfen önce geçerli bir masaya tıklayın!");
    return;
  }
  const basePath = `isletmeler/${aktifIsletme}`;
  const masaIdStr = String(seciliMasa.id);
  const hedefFirebaseKey = masaIdStr;

  const mevcutAdisyon = Array.isArray(seciliMasa.adisyon)
    ? [...seciliMasa.adisyon]
```

```
  : [];
const index = mevcutAdisyon.findIndex(
  (item) => String(item.id) === String(urun.id),
);

const ilkZaman =
  seciliMasa.adisyon && seciliMasa.adisyon.length > 0
    ? seciliMasa.eklenmeZamani || Date.now()
    : Date.now();

if (index > -1) {
  mevcutAdisyon[index] = {
    ...mevcutAdisyon[index],
    adet: Number(mevcutAdisyon[index].adet || 0) + 1,
  };
} else {
  mevcutAdisyon.push({
    id: String(urun.id || Date.now()),
    ad: urun.ad,
    fiyat: Number(urun.fiyat || 0),
    adet: 1,
    recete: urun.recete || [],
    eklenmeZamani: ilkZaman,
  });
}

const brutTutar = mevcutAdisyon.reduce(
```

```
  (toplam, item) => toplam + Number(item.adet || 0) * Number(item.fiyat || 0),
  0,
);
const aktifIskonto = Number(seciliMasa.iskonto ?? iskontoOrani ?? 0);
const netTutar = brutTutar * (1 - aktifIskonto / 100);
const masaAdi = seciliMasa.ad || `Masa ${hedefFirebaseKey}`;

const yeniMasaObjesi = {
  ...seciliMasa,
  id: hedefFirebaseKey,
  ad: masaAdi,
  adisyon: mevcutAdisyon,
  tutar: netTutar,
  brutTutar: brutTutar,
  durum: "dolu",
  not: seciliMasa.not !== undefined ? seciliMasa.not : masaNotu || "",
  iskonto: aktifIskonto,
  eklenmeZamani: ilkZaman,
};

setSeciliMasa(yeniMasaObjesi);
setMasalar((prev) =>
  prev.map((m) => (String(m.id) === hedefFirebaseKey ? yeniMasaObjesi : m)),
);

try {
  await set(ref(db, `${basePath}/masalar/${hedefFirebaseKey}`), yeniMasaObjesi);
```

```
    await set(ref(db, `${basePath}/siparisler/${hedefFirebaseKey}`), {
      id: hedefFirebaseKey,
      masaId: hedefFirebaseKey,
      masaNo: masaAdi,
      urunler: mevcutAdisyon,
      tutar: netTutar,
      durum: "bekliyor",
      eklenmeZamani: ilkZaman,
    });
  } catch (err) {
    console.error("Firebase kayıt hatası:", err);
  }
};

const urunAzalt = async (urunId: any) => {
  if (!seciliMasa || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const masaIdStr = String(seciliMasa.id);
  const hedefFirebaseKey = masaIdStr;

  const mevcutAdisyon = Array.isArray(seciliMasa.adisyon)
    ? [...seciliMasa.adisyon]
    : [];
  const index = mevcutAdisyon.findIndex((item) => String(item.id) === String(urunId));
  if (index === -1) return;

  if (mevcutAdisyon[index].adet > 1) {
```

```
  mevcutAdisyon[index] = {
    ...mevcutAdisyon[index],
    adet: mevcutAdisyon[index].adet - 1,
  };
} else {
  mevcutAdisyon.splice(index, 1);
}

const brutTutar = mevcutAdisyon.reduce(
  (toplam, item) => toplam + Number(item.adet || 0) * Number(item.fiyat || 0),
  0,
);
const netTutar = brutTutar * (1 - (iskontoOrani || 0) / 100);
const masaAdi = seciliMasa.ad || `Masa ${hedefFirebaseKey}`;
const bosKaldiMi = mevcutAdisyon.length === 0;

const yeniMasaObjesi = {
  ...seciliMasa,
  id: hedefFirebaseKey,
  ad: masaAdi,
  adisyon: mevcutAdisyon,
  tutar: netTutar,
  brutTutar: brutTutar,
  durum: bosKaldiMi ? "bos" : "dolu",
  not: masaNotu || "",
  iskonto: iskontoOrani || 0,
  eklenmeZamani: bosKaldiMi ? null : seciliMasa.eklenmeZamani,
```

```
  };

  setSeciliMasa(yeniMasaObjesi);
  setMasalar((prev) =>
    prev.map((m) => (String(m.id) === hedefFirebaseKey ? yeniMasaObjesi : m)),
  );

  await set(ref(db, `${basePath}/masalar/${hedefFirebaseKey}`), yeniMasaObjesi);
  if (bosKaldiMi) {
    await remove(ref(db, `${basePath}/siparisler/${hedefFirebaseKey}`)).catch(() => {});
  } else {
    await set(ref(db, `${basePath}/siparisler/${hedefFirebaseKey}`), {
      id: hedefFirebaseKey,
      masaId: hedefFirebaseKey,
      masaNo: masaAdi,
      urunler: mevcutAdisyon,
      durum: "bekliyor",
      eklenmeZamani: seciliMasa.eklenmeZamani,
    });
  }
};

const urunIptalEt = async (urunId: any) => {
  if (!seciliMasa || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const masaIdStr = String(seciliMasa.id);
  const hedefFirebaseKey = masaIdStr;
```

```
let mevcutAdisyon = Array.isArray(seciliMasa.adisyon)
  ? [...seciliMasa.adisyon]
  : [];
mevcutAdisyon = mevcutAdisyon.filter((item) => String(item.id) !== String(urunId));

const brutTutar = mevcutAdisyon.reduce(
  (toplam, item) => toplam + Number(item.adet || 0) * Number(item.fiyat || 0),
  0,
);
const netTutar = brutTutar * (1 - (iskontoOrani || 0) / 100);
const masaAdi = seciliMasa.ad || `Masa ${hedefFirebaseKey}`;
const bosKaldiMi = mevcutAdisyon.length === 0;

const yeniMasaObjesi = {
  ...seciliMasa,
  id: hedefFirebaseKey,
  ad: masaAdi,
  adisyon: mevcutAdisyon,
  tutar: netTutar,
  brutTutar: brutTutar,
  durum: bosKaldiMi ? "bos" : "dolu",
  not: masaNotu || "",
  iskonto: iskontoOrani || 0,
  eklenmeZamani: bosKaldiMi ? null : seciliMasa.eklenmeZamani,
};
```

```
  setSeciliMasa(yeniMasaObjesi);
  setMasalar((prev) =>
    prev.map((m) => (String(m.id) === hedefFirebaseKey ? yeniMasaObjesi : m)),
  );

  await set(ref(db, `${basePath}/masalar/${hedefFirebaseKey}`), yeniMasaObjesi);
  if (bosKaldiMi) {
    await remove(ref(db, `${basePath}/siparisler/${hedefFirebaseKey}`)).catch(() => {});
  } else {
    await set(ref(db, `${basePath}/siparisler/${hedefFirebaseKey}`), {
      id: hedefFirebaseKey,
      masaId: hedefFirebaseKey,
      masaNo: masaAdi,
      urunler: mevcutAdisyon,
      durum: "bekliyor",
      eklenmeZamani: seciliMasa.eklenmeZamani,
    });
  }
};

const iskontoUygula = async (oran: number) => {
  if (!seciliMasa || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  setIskontoOrani(oran);
  const brutTutar = (seciliMasa.adisyon || []).reduce(
    (acc: number, curr: any) => acc + curr.fiyat * curr.adet,
    0,
```

```
  );
  const netTutar = brutTutar * (1 - oran / 100);
  const guncelMasa = {
    ...seciliMasa,
    iskonto: oran,
    tutar: netTutar,
    brutTutar: brutTutar,
  };
  await update(ref(db, `${basePath}/masalar/${seciliMasa.id}`), guncelMasa);
  setSeciliMasa(guncelMasa);
};

const notKaydet = async () => {
  if (!seciliMasa || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const guncelMasa = { ...seciliMasa, not: masaNotu };
  await update(ref(db, `${basePath}/masalar/${seciliMasa.id}`), guncelMasa);
  setSeciliMasa(guncelMasa);
};

const hesabiKapat = async () => {
  if (!seciliMasa || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const masaIdStr = String(seciliMasa.id);

  try {
    const odenenTutar = Number(seciliMasa.tutar || 0);
```

```
const yeniKasa = {
  toplamCiro: (kasa.toplamCiro || 0) + odenenTutar,
  nakitToplam:
    odemeYontemi === "nakit"
      ? (kasa.nakitToplam || 0) + odenenTutar
      : kasa.nakitToplam || 0,
  kartToplam:
    odemeYontemi === "kart"
      ? (kasa.kartToplam || 0) + odenenTutar
      : kasa.kartToplam || 0,
  kapatilanAdisyonSayisi: (kasa.kapatilanAdisyonSayisi || 0) + 1,
};
await set(ref(db, `${basePath}/kasa`), yeniKasa);
await remove(ref(db, `${basePath}/siparisler/${masaIdStr}`)).catch(() => {});
await remove(ref(db, `${basePath}/siparisler/masa-${masaIdStr}`)).catch(() => {});

const bosMasaVerisi = {
  id: masaIdStr,
  ad: String(seciliMasa.ad || `Masa ${masaIdStr}`),
  durum: "bos",
  tutar: 0,
  brutTutar: 0,
  iskonto: 0,
  not: "",
  adisyon: [],
  eklenmeZamani: null,
};
```

```
    await set(ref(db, `${basePath}/masalar/${masaIdStr}`), bosMasaVerisi);
    setMasalar((prev) =>
      prev.map((m) => (String(m.id) === masaIdStr ? bosMasaVerisi : m)),
    );
    setSeciliMasa(bosMasaVerisi);
    alert("Hesap başarıyla kapatıldı ve veritabanı sıfırlandı.");
  } catch (err) {
    console.error("Hesap kapatma hatası:", err);
    alert("Hesap kapatılırken bir hata oluştu.");
  }
};

const adminUrunEkle = async (e: any) => {
  e.preventDefault();
  if (!yeniUrun.ad || !yeniUrun.fiyat || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const yeniUrunObj = {
    id: String(Date.now()),
    ad: yeniUrun.ad,
    fiyat: Number(yeniUrun.fiyat),
    kategori: yeniUrun.kategori,
    recete: [],
  };
  const guncelMenu = [...menu, yeniUrunObj];
  set(ref(db, `${basePath}/menu`), guncelMenu);
  setYeniUrun({ ad: "", fiyat: "", kategori: "Sıcak İçecekler" });
```

```
};

const masaEkle = async (e: any) => {
  e.preventDefault();
  if (!yeniMasaAdi || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  if (masalar.length >= mevcutLimitler.maxMasa) {
    alert(
      `Mevcut ${aktifPaket.toUpperCase()} paketinizde en fazla ${
        mevcutLimitler.maxMasa
      } masa tanımlayabilirsiniz!`,
    );
    return;
  }

  const yeniMasaId = String(masalar.length + 1);
  const yeniMasaObj = {
    id: yeniMasaId,
    ad: yeniMasaAdi,
    durum: "bos",
    tutar: 0,
    not: "",
    adisyon: [],
    eklenmeZamani: null,
  };

  await set(ref(db, `${basePath}/masalar/${yeniMasaId}`), yeniMasaObj);
```

```
  setYeniMasaAdi("");
};

const adminStokEkle = async (e: any) => {
  e.preventDefault();
  if (!yeniStok.ad || !yeniStok.miktar || !aktifIsletme) return;
  const basePath = `isletmeler/${aktifIsletme}`;
  const yeniStokObj = {
    id: String(Date.now()),
    ad: yeniStok.ad,
    miktar: Number(yeniStok.miktar),
    birim: yeniStok.birim,
    kritikMiktar: Number(yeniStok.kritikMiktar || 0),
  };
  const guncelStoklar = [...stoklar, yeniStokObj];
  set(ref(db, `${basePath}/stoklar`), guncelStoklar);
  setYeniStok({ ad: "", miktar: "", birim: "gr", kritikMiktar: "" });
};

if (superAdminModu) {
  return (
    <main className="min-h-screen bg-slate-900 text-white p-4 sm:p-8">
      <div className="max-w-5xl mx-auto space-y-6">
        <div className="flex justify-between items-center border-b border-slate-700 pb-4">
          <div>
            <h1 className="text-xl sm:text-2xl font-black text-amber-400">
              Süper Admin - İşletmeler Yönetimi
```

```
          </h1>
          <p className="text-xs text-slate-400 mt-0.5">
            Sistemdeki tüm kayıtlı kafe ve restoranlar ve paket yetkileri
          </p>
        </div>
        <button
          onClick={() => setSuperAdminModu(false)}
          className="bg-slate-800 hover:bg-slate-700 text-slate-200 px-3.5 py-2 rounded-xl text-xs font-bold border border-slate-700 cursor-pointer"
        >
          ← Giriş Ekranına Dön
        </button>
      </div>

      <div className="bg-slate-800 border border-slate-700 p-5 rounded-2xl shadow-xl">
        <h3 className="text-sm font-bold text-amber-400 mb-3">
          Yeni İşletme Kaydı Aç
        </h3>
        <form onSubmit={yeniIsletmeOlustur} className="grid grid-cols-1 sm:grid-cols-3 gap-3">
          <input
            type="text"
            placeholder="İşletme Kodu (örn: kahve-dunyasi)"
            value={yeniIsletmeAdi}
            onChange={(e) => setYeniIsletmeAdi(e.target.value)}
            className="bg-slate-900 border border-slate-700 p-2.5 rounded-xl text-xs text-white"
            required
          />
          <input
```

```
            type="password"
            placeholder="İşletme Şifresi"
            value={yeniIsletmeSifre}
            onChange={(e) => setYeniIsletmeSifre(e.target.value)}
            className="bg-slate-900 border border-slate-700 p-2.5 rounded-xl text-xs text-white"
            required
          />
          <button
            type="submit"
            className="bg-amber-500 hover:bg-amber-400 text-black font-bold py-2.5 rounded-xl text-xs cursor-pointer"
          >
            İşletme Ekle
          </button>
        </form>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
        {tumIsletmeler.length === 0 ? (
          <div className="col-span-full bg-slate-800/50 border border-slate-700 p-8 rounded-2xl text-center text-slate-400">
            Kayıtlı işletme bulunamadı veya veritabanı boş.
          </div>
        ) : (
          tumIsletmeler.map((isl) => {
            const paket = isl.lisans?.aktifPaket || "Trial";
            const paketStr = paket.toLowerCase();
            const bitis = isl.lisans?.bitisTarihi
```

```
            ? new Date(isl.lisans.bitisTarihi).toLocaleDateString("tr-TR")
            : "Belirtilmemiş";

          const kartRenkSiniflari = paketStr.includes("pro")
            ? "bg-emerald-950/40 border-emerald-500/60 text-emerald-100 shadow-emerald-500/10"
            : paketStr.includes("basic")
            ? "bg-amber-950/40 border-amber-500/60 text-amber-100 shadow-amber-500/10"
            : "bg-red-950/40 border-red-500/60 text-red-100 shadow-red-500/10";

          const etiketRenkSiniflari = paketStr.includes("pro")
            ? "bg-emerald-500/20 border border-emerald-500/50 text-emerald-300"
            : paketStr.includes("basic")
            ? "bg-amber-500/20 border border-amber-500/50 text-amber-300"
            : "bg-red-500/20 border border-red-500/50 text-red-300";

          return (
            <div
              key={isl.kod}
              className={`rounded-2xl p-5 shadow-xl flex flex-col justify-between space-y-4 border ${kartRenkSiniflari}`}
            >
              <div>
                <div className="flex justify-between items-start mb-2">
                  <h3 className={`text-lg font-black uppercase ${
                    paketStr.includes("pro") ? "text-emerald-400" : paketStr.includes("basic") ? "text-amber-400" : "text-red-400"
                  }`}>
                    {isl.kod}
```

```
                </h3>
                <span className={`text-[10px] px-2.5 py-0.5 rounded-full font-bold uppercase ${etiketRenkSiniflari}`}>
                  {paket}
                </span>
              </div>
              <p className="text-xs text-slate-400">
                Lisans Bitiş: <span className="text-slate-200">{bitis}</span>
              </p>
            </div>

            {/* Sadece Süper Admin buradan paket değiştirebilir */}
            <div className="space-y-2 pt-2 border-t border-slate-700/50">
              <div className="text-[10px] text-slate-400 font-bold">Paketi Değiştir:</div>
              <div className="grid grid-cols-3 gap-1">
                <button
                  onClick={async () => {
                    const bitisTarihi = new Date();
                    bitisTarihi.setDate(bitisTarihi.getDate() + 14);
                    await update(ref(db, `isletmeler/${isl.kod}/lisans`), {
                      aktifPaket: "Trial",
                      bitisTarihi: bitisTarihi.toISOString(),
                      durum: "aktif"
                    });
                  }}
                  className="bg-slate-800 hover:bg-slate-700 text-slate-200 py-1 rounded text-[10px] font-bold cursor-pointer"
                >
```

```
                Trial
              </button>
              <button
                onClick={async () => {
                  const bitisTarihi = new Date();
                  bitisTarihi.setFullYear(bitisTarihi.getFullYear() + 1);
                  await update(ref(db, `isletmeler/${isl.kod}/lisans`), {
                    aktifPaket: "Basic",
                    bitisTarihi: bitisTarihi.toISOString(),
                    durum: "aktif"
                  });
                }}
                className="bg-amber-900/60 hover:bg-amber-800 text-amber-200 py-1 rounded text-[10px] font-bold cursor-pointer"
              >
                Basic
              </button>
              <button
                onClick={async () => {
                  const bitisTarihi = new Date();
                  bitisTarihi.setFullYear(bitisTarihi.getFullYear() + 10);
                  await update(ref(db, `isletmeler/${isl.kod}/lisans`), {
                    aktifPaket: "Pro",
                    bitisTarihi: bitisTarihi.toISOString(),
                    durum: "aktif"
                  });
                }}
```

```
                  className="bg-emerald-900/60 hover:bg-emerald-800 text-emerald-200 py-1 rounded text-[10px] font-bold cursor-pointer"
                >
                  Pro
                </button>
              </div>

              <div className="flex gap-2 pt-2">
                <button
                  onClick={() => {
                    setAktifIsletme(isl.kod);
                    setSuperAdminModu(false);
                  }}
                  className="flex-1 bg-amber-500 hover:bg-amber-400 text-black font-bold py-2 rounded-xl text-xs cursor-pointer text-center transition-colors"
                >
                  İşletmeye Gir
                </button>
                <button
                  onClick={async () => {
                    if (confirm(`"${isl.kod}" işletmesini tamamen silmek istediğinize emin misiniz?`)) {
                      await remove(ref(db, `isletmeler/${isl.kod}`));
                    }
                  }}
                  className="bg-red-900/60 hover:bg-red-800 text-red-200 px-3 py-2 rounded-xl text-xs font-bold cursor-pointer transition-colors"
                  title="İşletmeyi Sil"
                >
                  Sil
```

```
                </button>
              </div>
            </div>
          </div>
        );
      })
    )}
  </div>
 </div>
</main>
);
}

if (!aktifIsletme) {
 return (
  <main className="min-h-screen bg-slate-900 text-white flex flex-col justify-center items-center p-4 relative py-12">
   <div className="absolute top-4 right-4">
    <button
     onClick={() => setSuperAdminModalAcik(true)}
     className="bg-slate-800 hover:bg-slate-700 text-cyan-400 border border-cyan-500/30 px-3.5 py-2 rounded-xl text-xs font-bold cursor-pointer shadow-lg"
    >
     Süper Admin Paneli
    </button>
   </div>

   <div className="flex flex-col items-center mb-6 text-center max-w-md w-full px-4">
```

```
        <h1 className="text-2xl sm:text-3xl font-extrabold text-amber-500 tracking-wide">
          POS & Adisyon Sistemi
        </h1>
        <p className="text-xs text-slate-400 mt-1">
          Lütfen işletme kodu ve şifrenizi girerek devam edin
        </p>
      </div>

      <div className="w-full max-w-md space-y-4">
        <form onSubmit={isletmeGirisYap} className="bg-slate-800 border border-slate-700 p-6 rounded-2xl shadow-xl space-y-4">
          <div>
            <label className="text-xs text-slate-400 block mb-1">İşletme Kodu</label>
            <input
              type="text"
              value={isletmeKodu}
              onChange={(e) => setIsletmeKodu(e.target.value)}
              placeholder="Örn: kahve-diyari"
              className="w-full bg-slate-900 border border-slate-700 p-3 rounded-xl text-white text-center text-sm focus:outline-none focus:border-amber-500"
              required
              autoFocus
            />
          </div>
          <div>
            <label className="text-xs text-slate-400 block mb-1">İşletme Şifresi</label>
            <input
              type="password"
```

```
            value={isletmeSifre}
            onChange={(e) => setIsletmeSifre(e.target.value)}
            placeholder="Şifrenizi giriniz..."
            className="w-full bg-slate-900 border border-slate-700 p-3 rounded-xl text-white text-center text-sm focus:outline-none focus:border-amber-500"
            required
          />
        </div>
        <button
          type="submit"
          className="w-full bg-amber-500 hover:bg-amber-400 text-black font-extrabold py-3 rounded-xl text-sm transition-all cursor-pointer shadow-lg"
        >
          İşletmeye Giriş Yap
        </button>
      </form>

      <button
        type="button"
        onClick={async () => {
          const demoKod = "demo-cafe";
          const demoSifre = "1234";
          const demoRef = ref(db, `isletmeler/${demoKod}`);
          const snapshot = await get(demoRef);
          if (!snapshot.exists()) {
            const defaultBitis = new Date();
            defaultBitis.setDate(defaultBitis.getDate() + 14);
            const hazirMenu = [
```

```
            { id: "1", ad: "Türk Kahvesi", fiyat: 60, kategori: "Sıcak İçecekler", recete: [] },
            { id: "2", ad: "Filtre Kahve", fiyat: 80, kategori: "Sıcak İçecekler", recete: [] },
            { id: "3", ad: "Çay", fiyat: 25, kategori: "Sıcak İçecekler", recete: [] },
            { id: "4", ad: "Taze Portakal Suyu", fiyat: 90, kategori: "Soğuk İçecekler", recete: [] },
            { id: "5", ad: "Kaşarlı Tost", fiyat: 110, kategori: "Yiyecekler", recete: [] },
            { id: "6", ad: "Cheesecake", fiyat: 130, kategori: "Tatlılar", recete: [] },
          ];
          await set(demoRef, {
            sifre: demoSifre,
            menu: hazirMenu,
            lisans: {
              aktifPaket: "Pro",
              bitisTarihi: defaultBitis.toISOString(),
              durum: "aktif",
            },
          });
        }
        setIsletmeKodu(demoKod);
        setIsletmeSifre(demoSifre);
        setAktifIsletme(demoKod);
      }}
      className="w-full bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-500 hover:to-blue-500 text-white font-extrabold py-3.5 rounded-2xl text-sm transition-all cursor-pointer shadow-xl flex items-center justify-center gap-2 border border-cyan-400/30"
    >
      <span>Tek Tıkla Demo (Test) Sürümü İle Giriş Yap</span>
    </button>
  </div>
```

```
{superAdminModalAcik && (
 <div className="fixed inset-0 bg-black/80 backdrop-blur-md flex justify-center items-center p-4 z-50">
  <div className="bg-slate-800 border border-slate-700 p-6 rounded-2xl max-w-sm w-full shadow-2xl text-center">
   <h3 className="text-lg font-bold text-amber-400 mb-4">Süper Admin Şifresi</h3>
   <form onSubmit={superAdminGirisKontrol} className="space-y-4">
    <div>
     <input
      type="password"
      value={superAdminSifreGiris}
      onChange={(e) => setSuperAdminSifreGiris(e.target.value)}
      placeholder="Şifrenizi giriniz..."
      className="w-full bg-slate-900 border border-slate-700 p-3 rounded-xl text-white text-center tracking-widest text-lg focus:outline-none focus:border-amber-500"
      autoFocus
      required
     />
    </div>
    <div className="flex gap-2">
     <button
      type="button"
      onClick={() => {
       setSuperAdminModalAcik(false);
       setSuperAdminSifreGiris("");
      }}
      className="w-1/2 bg-slate-700 hover:bg-slate-600 text-white font-bold py-2.5 rounded-xl text-xs transition-all cursor-pointer"
```

```
              >
                İptal
              </button>
              <button
                type="submit"
                className="w-1/2 bg-amber-500 hover:bg-amber-400 text-black font-extrabold py-2.5 rounded-xl text-xs transition-all cursor-pointer"
              >
                Giriş Yap
              </button>
            </div>
          </form>
        </div>
      </div>
    )}
  </main>
 );
}


if (!kullaniciRolu) {
 return (
  <main className="min-h-screen bg-slate-900 text-white flex flex-col justify-center items-center p-4 relative py-12">
    <div className="absolute top-4 left-4">
      <button
        onClick={() => setAktifIsletme(null)}
        className="text-xs bg-slate-800 hover:bg-slate-700 px-3 py-1.5 rounded-lg text-slate-300 font-bold border border-slate-700 cursor-pointer"
      >
```

```
          ← İşletme Değiştir ({aktifIsletme})
        </button>
      </div>

      {!lisansKilitli && kalanGun !== null && kalanGun <= 7 && (
        <div className="absolute top-0 left-0 right-0 bg-amber-500/20 border-b border-amber-500/30 text-amber-300 text-xs py-2 px-4 text-center font-bold">
          Demo kullanım sürenizin bitmesine <b>{kalanGun} gün</b> kaldı.
        </div>
      )}

      <div className="flex flex-col items-center mb-8 text-center max-w-md w-full px-4 mt-6">
        <h1 className="text-xl sm:text-3xl font-extrabold text-amber-500 tracking-wide uppercase">
          {aktifIsletme}
        </h1>
        <p className="text-xs text-slate-400 mt-1">Lütfen devam etmek için rolünüzü seçiniz</p>
      </div>

      <div className="grid grid-cols-1 sm:grid-cols-3 gap-4 sm:gap-6 max-w-xl w-full mb-8">
        <button
          type="button"
          onClick={() => {
            if (mevcutLimitler.maxGarson === 0) {
              alert("Trial pakette Garson hesabı yetkisi bulunmamaktadır. (Garson Limiti: 0)");
              return;
            }
            setKullaniciRolu("garson");
            setAktifSekme("salon");
```

```
        }}
        className="bg-slate-800/80 backdrop-blur border-2 border-amber-500/80 hover:border-amber-400 hover:bg-amber-600/20 p-5 sm:p-6 rounded-2xl flex flex-col items-center justify-center transition-all cursor-pointer shadow-lg"
      >
        <span className="font-bold text-base sm:text-lg">Garson Girişi</span>
        <span className="text-[10px] text-slate-400 mt-1">
          (Limit: {mevcutLimitler.maxGarson === Infinity ? "Sınırsız" : mevcutLimitler.maxGarson})
        </span>
      </button>

      <button
        type="button"
        onClick={() => {
          if (!mevcutLimitler.mutfakEkrani) {
            alert("Mutfak Ekranı yalnızca PRO pakette mevcuttur.");
            return;
          }
          setKullaniciRolu("mutfak");
          setAktifSekme("mutfak");
        }}
        className="bg-slate-800/80 backdrop-blur border-2 border-amber-500/80 hover:border-amber-400 hover:bg-amber-600/20 p-5 sm:p-6 rounded-2xl flex flex-col items-center justify-center transition-all cursor-pointer shadow-lg"
      >
        <span className="font-bold text-base sm:text-lg">Mutfak Ekranı</span>
      </button>

      <button
```

```
          type="button"
          onClick={() => setSifreModalAcik(true)}
          className="bg-slate-800/80 backdrop-blur border-2 border-cyan-500/80 hover:border-cyan-400 hover:bg-cyan-600/20 p-5 sm:p-6 rounded-2xl flex flex-col items-center justify-center transition-all cursor-pointer shadow-lg"
        >
          <span className="font-bold text-base sm:text-lg">Yönetici (Admin)</span>
        </button>
      </div>

      <div className="w-full max-w-xl bg-slate-800/60 border border-slate-700/80 rounded-2xl overflow-hidden shadow-xl mb-6">
        <button
          type="button"
          onClick={() => setAbonelikPanelAcik(!abonelikPanelAcik)}
          className="w-full p-4 text-left bg-slate-800 hover:bg-slate-700/80 transition-all flex justify-between items-center cursor-pointer"
        >
          <div>
            <h3 className="text-xs sm:text-sm font-bold text-amber-400">
              Abonelik Paketleri & Banka Bilgileri
            </h3>
            <p className="text-[10px] sm:text-[11px] text-slate-400">
              Paket detayları ve IBAN bilgileri için tıklayın.
            </p>
          </div>
          <span className="text-amber-400 font-bold">
            {abonelikPanelAcik ? "▲" : "▼"}
          </span>
```

```
        </button>
        {abonelikPanelAcik && (
          <div className="p-4 sm:p-5 border-t border-slate-700/80 space-y-4 bg-slate-900/60">
            <div className="grid grid-cols-1 sm:grid-cols-3 gap-3">
              <div className="bg-slate-800 p-3 rounded-xl border border-slate-700">
                <h4 className="text-xs font-bold text-white mt-1">Trial Sürüm</h4>
                <ul className="text-[10px] text-slate-400 mt-2 space-y-1">
                  <li>5 Masa Sınırı</li>
                  <li>1 Garson</li>
                </ul>
              </div>
              <div className="bg-slate-800 p-3 rounded-xl border border-blue-500/30">
                <h4 className="text-xs font-bold text-white mt-1">Basic Paket</h4>
                <p className="text-xs font-black text-amber-400 font-mono mt-0.5">1.000 TL/ay</p>
                <ul className="text-[10px] text-slate-300 mt-1.5 space-y-1">
                  <li>10 Masa Sınırı</li>
                  <li>2 Garson Hesabı</li>
                  <li>QR Kod Menü</li>
                </ul>
              </div>
              <div className="bg-gradient-to-br from-amber-500/10 to-slate-800 p-3 rounded-xl border border-amber-500/50">
                <h4 className="text-xs font-bold text-amber-400 mt-1">Pro Paket</h4>
                <p className="text-xs font-black text-amber-400 font-mono mt-0.5">2.000 TL/ay</p>
                <ul className="text-[10px] text-slate-200 mt-1.5 space-y-1">
                  <li>50 Masa Sınırı</li>
                  <li>Sınırsız Garson</li>
                  <li>QR Kod Menü</li>
```

```
            <li>Live Mutfak Ekranı</li>
            <li>Stok Yönetim Ekranı</li>
          </ul>
        </div>
      </div>

      {/* IBAN ve WhatsApp İletişim Bilgileri Geri Eklendi */}
      <div className="bg-slate-950/80 border border-slate-800 p-3 rounded-xl text-left">
        <span className="text-[9px] uppercase font-bold text-amber-400 block mb-0.5">
          Banka Hesap (IBAN) Bilgileri
        </span>
        <p className="text-xs font-mono font-bold text-white select-all">
          TR11 0004 904482759750 06
        </p>
        <p className="text-[11px] text-slate-300 mt-1">
          <b>Alıcı:</b> Mehmet Cihan Ferhatoğlu
        </p>
      </div>

      <div className="pt-1">
        <a
          href="https://wa.me/905000000000?text=Merhaba,%20POS%20ve%20Adisyon%20sistemi%20aboneliği%20hakkında%20bilgi%20almak%20istiyorum."
          target="_blank"
          rel="noopener noreferrer"
          className="w-full bg-emerald-600 hover:bg-emerald-500 text-white font-bold py-2.5 px-4 rounded-xl text-xs flex items-center justify-center gap-2 transition-all shadow-lg cursor-pointer"
        >
```

```
            <span>WhatsApp ile İletişime Geç</span>
          </a>
        </div>
      </div>
    )}
  </div>

  {sifreModalAcik && (
    <div className="fixed inset-0 bg-black/80 backdrop-blur-md flex justify-center items-center p-4 z-50">
      <div className="bg-slate-800 border border-slate-700 p-6 rounded-2xl max-w-sm w-full shadow-2xl text-center">
        <h3 className="text-lg font-bold text-amber-400 mb-4">Yönetici Girişi</h3>
        <form onSubmit={adminGirisYap} className="space-y-4">
          <div>
            <label className="text-xs text-slate-400 block mb-1">Admin / İşletme Şifresi</label>
            <input
              type="password"
              value={adminSifre}
              onChange={(e) => setAdminSifre(e.target.value)}
              placeholder="Şifrenizi giriniz..."
              className="w-full bg-slate-900 border border-slate-700 p-3 rounded-xl text-white text-center tracking-widest text-lg focus:outline-none focus:border-amber-500"
              autoFocus
            />
          </div>
          <div className="flex gap-2">
            <button
              type="button"
```

```
                onClick={() => setSifreModalAcik(false)}
                className="w-1/2 bg-slate-700 hover:bg-slate-600 text-white font-bold py-2.5 rounded-xl text-xs transition-all cursor-pointer"
              >
                İptal
              </button>
              <button
                type="submit"
                className="w-1/2 bg-amber-500 hover:bg-amber-400 text-black font-extrabold py-2.5 rounded-xl text-xs transition-all cursor-pointer"
              >
                Giriş Yap
              </button>
            </div>
          </form>
        </div>
      </div>
    )}
  </main>
 );
}


 const filtrelenmisMenu =
  seciliKategori === "Tümü"
   ? menu
   : menu.filter((u) => u.kategori === seciliKategori);


 const mutfakSiparisleri = (siparisler || [])
```

```
    .filter((s: any) => s && s.durum === "bekliyor" && s.urunler && s.urunler.length > 0)
    .sort((a: any, b: any) => (b.eklenmeZamani || 0) - (a.eklenmeZamani || 0));

  const maxMasaSayisi = mevcutLimitler?.maxMasa ?? 5;
  const siraliMasalar = Array.from({ length: maxMasaSayisi }, (_, index) => {
    const hedefId = String(index + 1);
    const kayitliMasa = (masalar || []).find((m: any) => String(m.id) === hedefId);

    return kayitliMasa || {
      id: hedefId,
      ad: `Masa ${hedefId}`,
      durum: "bos",
      tutar: 0,
      brutTutar: 0,
      iskonto: 0,
      not: "",
      adisyon: [],
      eklenmeZamani: null,
    };
  });

  const kritikStokSayisi = (stoklar || []).filter(
    (s: any) => s.miktar <= s.kritikMiktar,
  ).length;

  return (
    <main className="min-h-screen bg-slate-900 text-white p-3 sm:p-6 print:bg-white print:text-
black">
```

```
    <header className="flex flex-col gap-3 mb-6 border-b border-slate-700 pb-4 print:hidden">
      <div className="flex flex-wrap justify-between items-center gap-2">
        <div className="flex items-center gap-2 flex-wrap">
          <h1 className="text-xl sm:text-2xl font-extrabold text-amber-500 uppercase">
            {aktifIsletme}
          </h1>
          <span className="bg-slate-800 text-[10px] sm:text-xs px-2 py-0.5 rounded border border-slate-700 font-bold uppercase text-amber-400">
            Paket: {aktifPaket} ({mevcutLimitler.maxMasa} Masa)
          </span>
          <span className="bg-slate-800 text-[10px] sm:text-xs px-2 py-0.5 rounded border border-slate-700 font-bold uppercase text-cyan-400">
            {kullaniciRolu}
          </span>
        </div>
        <button
          onClick={() => setKullaniciRolu(null)}
          className="text-xs bg-red-900/50 hover:bg-red-800 px-2.5 py-1 rounded text-red-200 cursor-pointer font-bold"
        >
          Çıkış
        </button>
      </div>

      <div className="flex bg-slate-800 p-1 rounded-xl border border-slate-700 overflow-x-auto gap-1">
        {kullaniciRolu !== "mutfak" && (
          <button
            onClick={() => setAktifSekme("salon")}
```

```
          className={`px-3 py-1.5 rounded-lg font-bold text-xs whitespace-nowrap cursor-pointer ${
            aktifSekme === "salon" ? "bg-amber-500 text-black" : "text-slate-400"
          }`}
        >
          Masalar
        </button>
      )}
      <button
        onClick={() => {
          if (!mevcutLimitler.mutfakEkrani) {
            alert("Mutfak Ekranı yalnızca PRO pakette mevcuttur.");
            return;
          }
          setAktifSekme("mutfak");
        }}
        className={`px-3 py-1.5 rounded-lg font-bold text-xs whitespace-nowrap cursor-pointer relative ${
          aktifSekme === "mutfak" ? "bg-amber-500 text-black" : "text-slate-400"
        }`}
      >
        Mutfak {!mevcutLimitler.mutfakEkrani && "(PRO)"}
        {mutfakSiparisleri.length > 0 && mevcutLimitler.mutfakEkrani && (
          <span className="ml-1 bg-red-500 text-white text-[10px] px-1.5 py-0.2 rounded-full font-bold">
            {mutfakSiparisleri.length}
          </span>
        )}
      </button>
```

```
        {kullaniciRolu === "admin" && (
          <>
            <button
              onClick={() => setAktifSekme("stok")}
              className={`px-3 py-1.5 rounded-lg font-bold text-xs whitespace-nowrap cursor-pointer relative ${
                aktifSekme === "stok" ? "bg-amber-500 text-black" : "text-slate-400"
              }`}
            >
              Stok & Reçete
              {kritikStokSayisi > 0 && (
                <span className="ml-1 bg-red-500 text-white text-[10px] px-1.5 py-0.2 rounded-full font-bold">
                  !
                </span>
              )}
            </button>
            <button
              onClick={() => setAktifSekme("rapor")}
              className={`px-3 py-1.5 rounded-lg font-bold text-xs whitespace-nowrap cursor-pointer ${
                aktifSekme === "rapor" ? "bg-amber-500 text-black" : "text-slate-400"
              }`}
            >
              Rapor
            </button>
            <button
              onClick={() => setAktifSekme("admin")}
```

```
            className={`px-3 py-1.5 rounded-lg font-bold text-xs whitespace-nowrap cursor-pointer ${
              aktifSekme === "admin" ? "bg-amber-500 text-black" : "text-slate-400"
            }`}
           >
            Yönetim
           </button>
          </>
         )}
        </div>
      </header>

      {aktifSekme === "salon" && (
        <section className="print:hidden">
          <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-3 sm:gap-4">
            {siraliMasalar.map((masa) => {
              const doluMu = Array.isArray(masa.adisyon) && masa.adisyon.length > 0;
              const siparisZamani = masa.eklenmeZamani || null;
              return (
                <div
                  key={masa.id}
                  onClick={() => setSeciliMasa(masa)}
                  className={`p-4 sm:p-5 rounded-xl border-2 cursor-pointer transition-all hover:scale-105
shadow-lg relative ${
                    doluMu
                      ? "bg-red-950/40 border-red-500 text-red-200"
                      : "bg-emerald-950/40 border-emerald-500 text-emerald-100"
                  }`}
                 >
```

```
<div className="flex justify-between items-center mb-2">
 <span className="font-bold text-base sm:text-lg">{masa.ad}</span>
 {siparisZamani && doluMu && mevcutLimitler.sureSayaci && (
  <SiparisSuresi eklenmeZamani={siparisZamani} />
 )}
</div>
{mevcutLimitler.qrkod ? (
 <button
  type="button"
  onClick={(e) => {
   e.stopPropagation();
   setSeciliQrMasa(masa);
  }}
  className="mb-2 px-2.5 py-1 bg-amber-500 hover:bg-amber-400 text-black rounded-lg text-xs font-bold cursor-pointer"
 >
  QR Menü
 </button>
) : (
 <button
  type="button"
  onClick={(e) => {
   e.stopPropagation();
   alert("QR Kod özelliği Trial pakette bulunmamaktadır.");
  }}
  className="mb-2 opacity-50 cursor-not-allowed border border-dashed border-slate-700 text-slate-400 px-2 py-0.5 rounded-lg text-[10px]"
 >
```

```
              QR Menü (PRO)
            </button>
          )}
          {masa.not && (
            <p className="text-xs text-amber-300 italic truncate mb-2">
              Not: {masa.not}
            </p>
          )}
          <div className="text-right mt-3">
            <p className="text-xl sm:text-2xl font-extrabold">
              {Number(masa.tutar || 0).toFixed(2)} ₺
            </p>
          </div>
        </div>
      );
    })}
  </div>
 </section>
)}

{aktifSekme === "mutfak" && (
 <section className="grid grid-cols-1 md:grid-cols-3 gap-6 print:hidden">
  <div className="col-span-full grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
   {mutfakSiparisleri.length === 0 ? (
    <div className="col-span-full bg-slate-900 border border-slate-800 p-8 rounded-3xl text-center text-slate-400 shadow-xl">
     <p className="text-lg font-bold mb-1">Mutfakta Bekleyen Sipariş Yok</p>
     <p className="text-xs text-slate-400">Yeni siparişler geldiğinde burada görünecektir.</p>
```

```
          </div>
        ) : (
          mutfakSiparisleri.map((siparis: any) => (
            <div
              key={siparis.id}
              className="p-4 sm:p-5 rounded-3xl border transition-all shadow-xl flex flex-col justify-between bg-slate-900 border-slate-800"
            >
              <div>
                <div className="flex justify-between items-center mb-3 pb-3 border-b border-slate-800">
                  <span className="text-base font-black text-amber-400">
                    {siparis.masaNo || "Masa"}
                  </span>
                  <SiparisSuresi eklenmeZamani={siparis.eklenmeZamani} />
                </div>
                <ul className="space-y-2 mb-6">
                  {(siparis.urunler || []).map((urun: any, index: number) => (
                    <li
                      key={index}
                      className="flex justify-between items-center text-sm font-medium text-slate-200 bg-slate-800/50 p-2.5 rounded-xl border border-slate-700/50"
                    >
                      <span>{urun.ad}</span>
                      <span className="font-bold text-amber-400 bg-amber-400/10 px-2 py-0.5 rounded-lg text-xs">
                        {urun.adet} Adet
                      </span>
                    </li>
```

```
                ))}
              </ul>
            </div>
            <button
              type="button"
              onClick={async () => {
                const basePath = `isletmeler/${aktifIsletme}`;
                await remove(ref(db, `${basePath}/siparisler/${siparis.id}`));
                await update(ref(db, `${basePath}/masalar/${siparis.masaId || siparis.id}`), {
                  eklenmeZamani: null,
                });
                calSiparisSesi();
              }}
              className="w-full px-4 py-3 bg-emerald-600 hover:bg-emerald-500 text-white rounded-xl font-bold text-xs transition-all cursor-pointer flex items-center justify-center gap-2 shadow-lg"
            >
              Siparişi Hazır Yap & Düşür
            </button>
          </div>
        ))
      )}
    </div>
  </section>
)}

{aktifSekme === "stok" && kullaniciRolu === "admin" && (
  <section className="max-w-5xl mx-auto space-y-6">
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
```

```
        <div className="md:col-span-2 bg-slate-800 p-4 sm:p-5 rounded-2xl border border-slate-700">
          <h3 className="text-lg font-bold text-amber-400 mb-4">Depo Ham Madde Stokları</h3>
          <div className="overflow-x-auto">
            <table className="w-full text-left text-xs text-slate-300">
              <thead className="bg-slate-900 uppercase text-slate-400">
                <tr>
                  <th className="p-2">Malzeme</th>
                  <th className="p-2">Miktar</th>
                  <th className="p-2">Birim</th>
                  <th className="p-2">Kritik Limit</th>
                  <th className="p-2">Durum</th>
                </tr>
              </thead>
              <tbody>
                {(stoklar || []).map((item) => {
                  const isKritik = item.miktar <= item.kritikMiktar;
                  return (
                    <tr key={item.id} className="border-b border-slate-700">
                      <td className="p-2 font-bold text-white">{item.ad}</td>
                      <td className="p-2 font-extrabold text-amber-400">{item.miktar}</td>
                      <td className="p-2">{item.birim}</td>
                      <td className="p-2">{item.kritikMiktar}</td>
                      <td className="p-2">
                        {isKritik ? (
                          <span className="bg-red-900/80 text-red-200 px-2 py-0.5 rounded text-[10px] font-bold">
                            Stok Az!
```

```
                    </span>
                  ) : (
                    <span className="bg-emerald-900/80 text-emerald-200 px-2 py-0.5 rounded text-[10px] font-bold">
                      Yeterli
                    </span>
                  )}
                </td>
              </tr>
            );
          })}
        </tbody>
      </table>
    </div>
  </div>

  <div className="bg-slate-800 p-4 sm:p-5 rounded-2xl border border-slate-700">
    <h3 className="text-md font-bold text-amber-400 mb-3">Ham Madde Ekle</h3>
    <form onSubmit={adminStokEkle} className="space-y-3">
      <div>
        <label className="text-[10px] text-slate-400">Malzeme Adı</label>
        <input
          type="text"
          value={yeniStok.ad}
          onChange={(e) => setYeniStok({ ...yeniStok, ad: e.target.value })}
          placeholder="Örn: Kahve Çekirdeği"
          className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
        />
```

```
</div>
<div className="grid grid-cols-2 gap-2">
 <div>
  <label className="text-[10px] text-slate-400">Miktar</label>
  <input
   type="number"
   value={yeniStok.miktar}
   onChange={(e) => setYeniStok({ ...yeniStok, miktar: e.target.value })}
   placeholder="1000"
   className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
  />
 </div>
 <div>
  <label className="text-[10px] text-slate-400">Birim</label>
  <select
   value={yeniStok.birim}
   onChange={(e) => setYeniStok({ ...yeniStok, birim: e.target.value })}
   className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
  >
   <option value="gr">Gram (gr)</option>
   <option value="ml">Mililitre (ml)</option>
   <option value="adet">Adet</option>
   <option value="kg">Kilogram (kg)</option>
  </select>
 </div>
</div>
<div>
```

```
            <label className="text-[10px] text-slate-400">Kritik Uyarı Limiti</label>
            <input
              type="number"
              value={yeniStok.kritikMiktar}
              onChange={(e) => setYeniStok({ ...yeniStok, kritikMiktar: e.target.value })}
              placeholder="200"
              className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
            />
           </div>
           <button className="w-full bg-amber-600 hover:bg-amber-500 py-2 rounded text-xs font-bold cursor-pointer">
             Stok Kaydet
           </button>
          </form>
         </div>
        </div>
       </section>
     )}

     {aktifSekme === "rapor" && kullaniciRolu === "admin" && (
       <section className="max-w-2xl mx-auto bg-slate-800 p-5 sm:p-6 rounded-2xl border border-slate-700">
        <h2 className="text-lg sm:text-xl font-bold mb-4 text-amber-400">Gün Sonu Kasası</h2>
        <div className="space-y-3">
         <div className="flex justify-between border-b border-slate-700 pb-2 text-sm sm:text-base">
          <span>Toplam Ciro:</span>
          <span className="font-bold text-emerald-400 text-lg sm:text-xl">
           {kasa.toplamCiro?.toFixed(2) || 0}₺
```

```
      </span>
    </div>
    <div className="flex justify-between border-b border-slate-700 pb-2 text-xs sm:text-sm">
      <span>Nakit Tahsilat:</span>
      <span className="font-bold text-amber-400">
        {kasa.nakitToplam?.toFixed(2) || 0}₺
      </span>
    </div>
    <div className="flex justify-between text-xs sm:text-sm">
      <span>Kredi Kartı Tahsilat:</span>
      <span className="font-bold text-cyan-400">
        {kasa.kartToplam?.toFixed(2) || 0}₺
      </span>
    </div>
  </div>
 </section>
)}

{aktifSekme === "admin" && kullaniciRolu === "admin" && (
 <section className="max-w-4xl mx-auto space-y-6">
  <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
    <div className="bg-slate-800 p-5 rounded-2xl border border-slate-700">
      <h3 className="text-md font-bold text-cyan-400 mb-3">Yeni Menü Ürünü Ekle</h3>
      <form onSubmit={adminUrunEkle} className="space-y-3">
        <div>
          <label className="text-xs text-slate-400">Ürün Adı</label>
          <input
```

```
    type="text"
    value={yeniUrun.ad}
    onChange={(e) => setYeniUrun({ ...yeniUrun, ad: e.target.value })}
    placeholder="Örn: Filtre Kahve"
    className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
   />
  </div>
  <div>
   <label className="text-xs text-slate-400">Fiyat (₺)</label>
   <input
    type="number"
    value={yeniUrun.fiyat}
    onChange={(e) => setYeniUrun({ ...yeniUrun, fiyat: e.target.value })}
    placeholder="85"
    className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
   />
  </div>
  <div>
   <label className="text-xs text-slate-400">Kategori</label>
   <select
    value={yeniUrun.kategori}
    onChange={(e) => setYeniUrun({ ...yeniUrun, kategori: e.target.value })}
    className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
   >
    {kategoriler
     .filter((k) => k !== "Tümü")
     .map((k) => (
```

```
                <option key={k} value={k}>{k}</option>
              ))}
            </select>
          </div>
          <button className="w-full bg-cyan-600 hover:bg-cyan-500 py-2 rounded font-bold text-xs cursor-pointer">
            Ürün Kaydet
          </button>
        </form>
      </div>

      <div className="bg-slate-800 p-5 rounded-2xl border border-slate-700">
        <h3 className="text-md font-bold text-amber-400 mb-3">Mevcut Menü Ürünleri</h3>
        <div className="space-y-2 max-h-60 overflow-y-auto pr-1">
          {menu.length === 0 ? (
            <p className="text-xs text-slate-500">Menüde henüz ürün bulunmuyor.</p>
          ) : (
            menu.map((u) => (
              <div
                key={u.id}
                className="bg-slate-900 p-2.5 rounded-xl border border-slate-700 flex justify-between items-center"
              >
                <div>
                  <p className="text-xs font-bold text-white">{u.ad}</p>
                  <span className="text-[10px] text-slate-400 bg-slate-800 px-2 py-0.5 rounded">
                    {u.kategori} - {u.fiyat} ₺
                  </span>
```

```
                </div>
                <button
                  type="button"
                  onClick={() => urunSil(u.id)}
                  className="bg-red-900/60 hover:bg-red-800 text-red-200 px-2.5 py-1 rounded-lg text-xs font-bold cursor-pointer"
                >
                  Sil
                </button>
              </div>
            ))
          )}
        </div>
      </div>

      <div className="bg-slate-800 p-5 rounded-2xl border border-slate-700">
        <h3 className="text-md font-bold text-amber-400 mb-3">Yeni Masa Ekle</h3>
        <form onSubmit={masaEkle} className="space-y-3">
          <div>
            <label className="text-xs text-slate-400">
              Masa Adı (Limit: {mevcutLimitler.maxMasa} Masa)
            </label>
            <input
              type="text"
              value={yeniMasaAdi}
              onChange={(e) => setYeniMasaAdi(e.target.value)}
              placeholder="Örn: Masa 8"
              className="w-full bg-slate-900 p-2 rounded border border-slate-700 text-xs text-white"
```

```
            />
          </div>
          <button className="w-full bg-amber-600 hover:bg-amber-500 py-2 rounded font-bold text-xs cursor-pointer">
            Masa Oluştur
          </button>
        </form>
      </div>
    </div>
  </section>
 )}

 {seciliMasa && (
  <div className="fixed inset-0 bg-black/80 backdrop-blur-sm flex justify-center items-center p-2 sm:p-4 z-50 overflow-y-auto">
    <div className="bg-slate-800 border border-slate-700 rounded-2xl w-full max-w-2xl p-3 sm:p-6 relative my-auto max-h-[95vh] flex flex-col">
      <div className="flex justify-between items-center border-b border-slate-700 pb-2 mb-3">
        <h3 className="text-base sm:text-xl font-bold text-amber-400">
          {seciliMasa.ad}
        </h3>
        <button
          onClick={() => setSeciliMasa(null)}
          className="text-lg sm:text-2xl cursor-pointer text-slate-400 hover:text-white font-bold p-1"
        >
          ×
        </button>
      </div>
```

```
<div className="grid grid-cols-1 sm:grid-cols-2 gap-3 sm:gap-4 overflow-y-auto pr-0.5">
 <div>
  <div className="flex gap-1 overflow-x-auto pb-2 mb-2 no-scrollbar">
   {kategoriler.map((k) => (
    <button
     key={k}
     onClick={() => setSeciliKategori(k)}
     className={`text-[10px] sm:text-xs px-2 py-1 rounded whitespace-nowrap cursor-pointer ${
      seciliKategori === k
       ? "bg-amber-500 text-black font-bold"
       : "bg-slate-700 text-slate-300"
     }`}
    >
     {k}
    </button>
   ))}
  </div>
  <div className="space-y-1.5 max-h-40 sm:max-h-60 overflow-y-auto pr-1">
   {filtrelenmisMenu.map((u) => (
    <button
     key={u.id}
     onClick={() => urunEkle(u)}
     className="w-full text-left bg-slate-700/80 hover:bg-amber-600 p-2 rounded-lg text-xs flex justify-between items-center cursor-pointer transition-colors"
    >
     <span className="font-medium text-white truncate mr-2">{u.ad}</span>
```

```
              <span className="font-bold text-amber-300 whitespace-nowrap">{u.fiyat} ₺</span>
            </button>
          ))}
        </div>
      </div>

      <div className="bg-slate-900/90 p-3 rounded-xl flex flex-col justify-between space-y-3 border border-slate-700/50">
        <ul className="space-y-1.5 text-xs max-h-40 sm:max-h-52 overflow-y-auto pr-1">
          {(seciliMasa.adisyon || []).map((item: any) => (
            <li
              key={item.id}
              className="flex justify-between items-center bg-slate-800 p-2 rounded-lg border border-slate-700 gap-1.5"
            >
              <div className="min-w-0 flex-1">
                <p className="font-bold text-xs text-white truncate">{item.ad}</p>
                <p className="text-[10px] text-amber-400 font-semibold">
                  {item.adet} x {item.fiyat} = {(item.adet * item.fiyat).toFixed(2)} ₺
                </p>
              </div>
              <div className="flex items-center gap-1 flex-shrink-0">
                <button
                  type="button"
                  onClick={() => urunAzalt(item.id)}
                  className="w-6 h-6 bg-slate-700 hover:bg-slate-600 text-white font-bold rounded cursor-pointer text-xs flex items-center justify-center"
                >
                  -
```

```
                </button>
                <span className="text-xs font-bold px-1 text-slate-200">{item.adet}</span>
                <button
                  type="button"
                  onClick={() => urunEkle(item)}
                  className="w-6 h-6 bg-amber-500 hover:bg-amber-400 text-black font-bold rounded cursor-pointer text-xs flex items-center justify-center"
                >
                  +
                </button>
                <button
                  type="button"
                  onClick={() => urunIptalEt(item.id)}
                  className="w-6 h-6 bg-red-900/80 hover:bg-red-700 text-white font-bold rounded cursor-pointer text-xs flex items-center justify-center ml-0.5"
                  title="Ürünü İptal Et"
                >
                  🗑
                </button>
              </div>
            </li>
          ))}
        </ul>

        <div className="space-y-2 pt-1">
          <div className="flex gap-1.5">
            <input
              type="text"
```

```
              value={masaNotu}
              onChange={(e) => setMasaNotu(e.target.value)}
              placeholder="Masa notu ekle..."
              className="w-full bg-slate-800 p-1.5 rounded-lg border border-slate-700 text-xs text-white placeholder-slate-500 focus:outline-none focus:border-amber-500"
            />
            <button
              type="button"
              onClick={notKaydet}
              className="bg-slate-700 hover:bg-slate-600 px-3 py-1.5 rounded-lg text-xs font-bold cursor-pointer text-slate-200"
            >
              Kaydet
            </button>
          </div>

          <div className="flex flex-wrap items-center justify-between text-xs gap-1">
            <span className="text-slate-400 font-medium">İskonto:</span>
            <div className="flex flex-wrap gap-1">
              {[0, 10, 20, 50].map((oran) => (
                <button
                  key={oran}
                  type="button"
                  onClick={() => iskontoUygula(oran)}
                  className={`px-2 py-1 rounded-lg cursor-pointer text-[10px] sm:text-xs transition-colors ${
                    iskontoOrani === oran
                      ? "bg-amber-500 text-black font-bold"
                      : "bg-slate-800 text-slate-300 hover:bg-slate-700"
```

```
          }`}
        >
          %{oran}
        </button>
      ))}
    </div>
  </div>
</div>

<div className="border-t border-slate-700 pt-2 space-y-2">
  <div className="flex justify-between font-extrabold text-sm sm:text-base">
    <span>TOPLAM:</span>
    <span className="text-amber-400">
      {Number(seciliMasa.tutar || 0).toFixed(2)} ₺
    </span>
  </div>
  <div className="flex gap-2">
    <button
      type="button"
      onClick={() => setOdemeYontemi("nakit")}
      className={`w-1/2 py-1.5 rounded-lg text-xs font-bold cursor-pointer transition-colors
${
        odemeYontemi === "nakit" ? "bg-amber-500 text-black" : "bg-slate-800 text-slate-400"
      }`}
    >
      Nakit
    </button>
    <button
```

```
                type="button"
                onClick={() => setOdemeYontemi("kart")}
                className={`w-1/2 py-1.5 rounded-lg text-xs font-bold cursor-pointer transition-colors
${
                  odemeYontemi === "kart" ? "bg-cyan-500 text-black" : "bg-slate-800 text-slate-400"
                }`}
              >
                Kart
              </button>
            </div>
            <button
              type="button"
              onClick={hesabiKapat}
              className="w-full bg-emerald-600 hover:bg-emerald-500 py-2.5 rounded-xl text-xs font-bold cursor-pointer shadow-lg transition-all"
            >
              Öde & Hesabı Kapat
            </button>
          </div>
        </div>
      </div>
    </div>
  )}

  {seciliQrMasa && (
    <div className="fixed inset-0 bg-black/85 backdrop-blur-sm z-50 flex items-center justify-center p-4">
```

```
        <div className="bg-slate-800 border border-slate-700 p-6 rounded-3xl max-w-sm w-full text-center relative shadow-2xl">
          <button
            onClick={() => setSeciliQrMasa(null)}
            className="absolute top-4 right-4 text-slate-400 hover:text-white font-bold text-xl cursor-pointer"
          >
            ×
          </button>
          <h3 className="text-xl font-black text-amber-400 mb-1">{seciliQrMasa.ad}</h3>
          <p className="text-[10px] text-amber-300 mb-2 break-all bg-slate-900 p-2 rounded border border-slate-700 font-mono">
            https://pos-adisyon-system.vercel.app/qr?isletme={aktifIsletme}&masa={seciliQrMasa.id}
          </p>
          <div className="bg-white p-4 rounded-2xl inline-block border-4 border-amber-500/30 mb-4">
            <img
              src={`https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=${encodeURIComponent(
                `https://pos-adisyon-system.vercel.app/qr?isletme=${aktifIsletme}&masa=${seciliQrMasa.id}`,
              )}`}
              alt="QR Kodu"
              className="w-48 h-48 mx-auto"
            />
          </div>
          <button
            type="button"
            onClick={() => setSeciliQrMasa(null)}
            className="w-full bg-amber-500 text-black font-bold py-2 rounded-xl text-xs cursor-pointer"
```

```
          >
            Kapat
          </button>
        </div>
      </div>
    )}
  </main>
 );
}
```